

---

# Hierarchical Models and Partial Pooling

ECON 8310: Business Forecasting — Lecture 11, Part 2

---

Department of Economics

University of Nebraska at Omaha

August 25, 2026

# Lecture Outline

---

- 1 Which Series May Be Pooled
- 2 Partial Pooling
- 3 Fitting It, and the Trap
- 4 Reading the Result
- 5 Key Takeaways

---

## **Which Series May Be Pooled**

Before you pool anything, ask what the groups have in common.

---

## The Question

---

A concrete one, and you have met it before. **Does demand rise in weeks when SNAP benefits are disbursed, and by how much — store by store?** Homework 3's memo had you stock up for SNAP weeks store by store, which assumed those differences are real. Today we check.

**But one question comes before the model.** SNAP is a *food* assistance programme. Measure the raw SNAP-week effect on log units in all thirty series, grouped by category:

| Category                 | Mean effect | Spread across stores |
|--------------------------|-------------|----------------------|
| <b>FOODS</b> (10 series) | +0.089      | 0.058                |
| HOBBIES (10 series)      | +0.006      | 0.010                |
| HOUSEHOLD (10 series)    | +0.011      | 0.014                |

**These are not thirty versions of the same thing.** FOODS responds; the other two barely move. Pooling all thirty would tell the model they are interchangeable.

## Exchangeability: the Assumption Under Every Hierarchy

---

Partial pooling rests on one assumption, worth naming before we use it.

[  
Exchangeable] Groups are **exchangeable** when, before seeing the data, you have no reason to expect one to differ from another in a particular direction. Swapping their labels would not change what you believe.

The ten FOODS series qualify: same category, different stores, and you could not say in advance which store's SNAP effect should be largest. FOODS against HOBBIES does **not** — you know before looking that a food-assistance programme moves food.

**The setup for today.** We work with the **ten FOODS series**, and deliberately thin **five** of them down to **8 weeks**. Five series are data-rich, five are data-poor. That imbalance is the whole point.

*The lab pools all thirty anyway, and measures what it costs.*

## Complete Pooling and No Pooling

---

Within that exchangeable set there are two obvious approaches, and each fails in a different direction.

|                         | What it does                                                                    | How it fails                                                                                    |
|-------------------------|---------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <b>Complete pooling</b> | One SNAP effect for all ten series.<br><code>b = pm.Normal("b", 0, 1)</code>    | Denies that stores differ at all. A genuinely different store is averaged away.                 |
| <b>No pooling</b>       | A separate effect per series, estimated independently.<br><code>shape=10</code> | A series with 8 weeks gets an estimate built from 8 weeks. Mostly noise, reported as a finding. |

**You have used both already without naming them.** Homework 3's random forest pooled all thirty series into one model with store dummies — close to complete pooling. Homework 1 fitted ARIMA to one series at a time — no pooling. The choice has always been made by convenience; partial pooling makes it a modelling decision and lets the data settle it.

---

## Partial Pooling

Let each series have its own effect — drawn from a shared distribution the data also estimates.

---

## The Hierarchy Is One Extra Line

---

Give each series its own effect  $b_j$ , but say where those effects come from:

$$b_j \sim \mathcal{N}(\mu_b, \sigma_b), \quad j = 1, \dots, 10$$

$\mu_b$  is the typical SNAP effect across FOODS, and  $\sigma_b$  is **how much stores genuinely differ**. Crucially, both are estimated from the data rather than assumed.

---

|                          |                                                                                                  |
|--------------------------|--------------------------------------------------------------------------------------------------|
| $\sigma_b \rightarrow 0$ | The model concludes stores do not differ, and reproduces complete pooling.                       |
| $\sigma_b$ large         | The model concludes they differ a lot, and approaches no pooling.                                |
| In between               | Each $b_j$ is pulled toward $\mu_b$ — <i>how far</i> depending on how much data that series has. |

---

**That last row is the whole idea.** A series with 277 weeks barely moves; one with 8 weeks moves a long way. Nobody chose those weights — they fall out of the model.



## Why This Is the Right Answer, Not a Compromise

---

Partial pooling sounds like splitting the difference between two bad options. It is better than that, and the reason is worth stating.

When a series has 8 observations, its unpooled estimate is mostly sampling noise. The hierarchy's pull toward  $\mu_b$  is not a fudge — it is the correct response to a noisy measurement. You already believe this in another form: it is why you would not conclude a coin is biased after three flips.

**This is regularization again, for the sixth time.** A hierarchical prior shrinks per-group estimates toward a common value exactly as Ridge shrinks coefficients toward zero — and here the shrinkage strength  $\sigma_b$  is *estimated* rather than tuned by cross-validation.

*GAM smoothing penalty (L3) → tree pruning (L4) → `reg_lambda` (L5) → Ridge and LASSO (L6) → weight decay (L7) → hierarchical priors (today).*

---

## **Fitting It, and the Trap**

The obvious code does not work, and the reason is geometry rather than statistics.

---

## Non-centred hierarchical model

```
with pm.Model() as model:

    mu_b = pm.Normal("mu_b", 0, 0.5)      # typical SNAP effect
    sd_b = pm.HalfNormal("sd_b", 0.25)    # how much stores differ
    z     = pm.Normal("z", 0, 1, shape=J)  # non-centred
    b     = pm.Deterministic("b", mu_b + z * sd_b)

    a = pm.Normal("a", y.mean(), 1, shape=J) # per-series level
    s = pm.Exponential("s", 1.0)

    pm.Normal("obs", mu=a[idx] + b[idx]*snap, sigma=s, observed=y)
```

`idx` maps each row to its series, so `b[idx]` picks out the right effect for every observation. That indexing pattern is how every hierarchical model in PyMC is written.

**Note what is *not* here.** No decision about how much to pool, no weighting scheme, no threshold for “enough data.” Those all come out of estimating  $\sigma_b$ .

## The Trap: Write It the Obvious Way and It Breaks

The natural way to write the hierarchy is `b = pm.Normal("b", mu_b, sd_b, shape=J)` — the *centred* form, which reads exactly like the equation. On our data it produces:

|             | Centred   | Non-centred | Want   |
|-------------|-----------|-------------|--------|
| Divergences | <b>43</b> | <b>0</b>    | 0      |
| $\hat{R}$   | 1.003     | 1.003       | < 1.01 |
| ESS (bulk)  | 966       | 1,719       | > 400  |

**Why:** when  $\sigma_b$  is small, the  $b_j$  must crowd into a narrow band, and the posterior takes on a funnel shape — wide where  $\sigma_b$  is large, pinched where it is small. A sampler tuned for the wide part cannot navigate the neck, and reports divergences.

**Note which alarm actually fired.**  $\hat{R}$  and ESS *both pass* — on those two numbers you would have shipped it. Only the divergence count objects. On hierarchical models divergences are the sensitive instrument; do not wait for  $\hat{R}$  to tell you.

*BMCP §4.6.1, Posterior Geometry Matters — the single most common failure in applied hierarchical modelling.*

---

## Reading the Result

Shrinkage, measured — and checked against data the model never saw.

---

## Shrinkage, Measured

Fit both models. Compare each series' estimated SNAP effect before and after pooling.

| Series group                       | Unpooled spread | After pooling | Mean move    |
|------------------------------------|-----------------|---------------|--------------|
| <b>Thin</b> — 8 weeks (5 series)   | 0.108           | 0.010         | <b>0.120</b> |
| <b>Full</b> — 277 weeks (5 series) | 0.055           | 0.037         | 0.015        |

**The data-poor series moved 8.1 times further toward the group mean than the data-rich ones.** Nobody told the model which series were thin. It inferred that from how much each series had to say.

Read the two right-hand columns together. The thin series' apparent spread of 0.108 nearly vanishes, while the full series **keep most of theirs** ( $0.055 \rightarrow 0.037$ ). The hierarchy did not flatten everything: it discarded spread that 8 weeks could not support and kept spread that 277 weeks could.

**That is the claim to be sceptical of, so the next slide tests it.**

## Checked Against Data the Model Never Saw

We thinned those five series ourselves, so the weeks we removed are still there. Compare each estimate against the SNAP effect actually observed in the held-out weeks.

| Thin series               | Unpooled     | Hierarchical | Held-out truth |
|---------------------------|--------------|--------------|----------------|
| CA_1_FOODS                | 0.286        | 0.097        | 0.087          |
| CA_3_FOODS                | 0.105        | 0.082        | 0.102          |
| TX_1_FOODS                | 0.052        | 0.075        | 0.082          |
| TX_3_FOODS                | 0.219        | 0.091        | 0.072          |
| WI_2_FOODS                | 0.338        | 0.103        | 0.207          |
| <b>RMSE against truth</b> | <b>0.126</b> | <b>0.048</b> | —              |

**Partial pooling was two and a half times more accurate**, beating the unpooled fit on four series out of five. Those estimates were not just noisy — they were confidently wrong, and the confidence came from 8 weeks of data.

*Borrowing strength is not a figure of speech: the thin series were estimated better because the other five existed.*

## What the Model Actually Concluded

---

The estimated hierarchy:  $\mu_b = 0.078$  with a 94% HDI of  $[0.017, 0.150]$ , and  $\sigma_b = 0.065$ .

In words: a SNAP week lifts FOODS units by roughly **8%**, and stores genuinely differ around that by about **7 percentage points**. Both numbers are answers, and the second one is the one no point estimate would have given you.

**It does not license the store-level policy the unpooled fit suggested.** An analyst reading the unpooled numbers would report uplifts from 5% to 40% and stock accordingly. The held-out weeks say that range was mostly noise — but  $\sigma_b = 0.065$  says it is not *all* noise either.

**The defensible recommendation:** plan on an 8% FOODS uplift everywhere, allow real but modest store variation around it, and do not act on a store's own estimate until that store has the weeks to support one.



---

## Key Takeaways

What hierarchy buys, and what it costs.

---

## Lecture 11 Part 2: Key Takeaways

---

1. **Ask what may be pooled before you pool it.** Groups must be **exchangeable**. FOODS and HOBBIES are not, and no amount of good sampling fixes a hierarchy built over the wrong set.
2. **Complete pooling** denies groups differ; **no pooling** lets a group with 8 observations report noise as a finding. You have used both without naming them.
3. **Partial pooling** gives each group its own parameter drawn from a shared distribution, and estimates how much groups genuinely differ.
4. **Shrinkage is automatic and proportionate.** Our thin series moved  $8.1\times$  further toward the group mean than data-rich ones, with nobody specifying that — and on held-out weeks it was  $2.6\times$  more accurate.
5. This is **regularization for the sixth time** — and the only version where the shrinkage strength is estimated rather than tuned.
6. **Write it non-centred.** The obvious form gave 43 divergences; the reparameterized form gave zero. Same model, different geometry — and divergences were the *only* diagnostic that objected.

*Next: Bayesian linear regression — coefficients as distributions, and what you can ask them.*

# References I

---